



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1 по дисциплине «Анализ алгоритмов»

Тема Алгоритмы умножения матриц

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2025

Содержание

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Стандартный алгоритм умножения матриц	4
1.2 Алгоритм Винограда	4
1.3 Алгоритм Винограда с оптимизацией	5
2 Конструкторская часть	6
2.1 Описание алгоритмов	6
2.2 Расчёт трудоёмкости алгоритмов	9
3 Технологическая часть	11
3.1 Реализация алгоритмов	11
3.2 Тестирование	13
4 Исследовательская часть	14
4.1 Замеры процессорного времени	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16

ВВЕДЕНИЕ

Цель работы – сравнить алгоритмы умножения матриц.

Задачи, необходимые для достижения поставленной цели:

- 1) описать алгоритмы умножения матриц;
- 2) ввести модель вычислений и оценить трудоёмкость каждого алгоритма в введённой модели;
- 3) реализовать алгоритмы;
- 4) описать архитектуру ЭВМ и выполнить замеры процессорного времени для лучшего и худшего случаев;
- 5) проверить соответствие теоретической оценки полученным значениям;
- 6) провести сравнительный анализ реализаций по трудоёмкости.

1 Аналитическая часть

Матрица – прямоугольная таблица из чисел, содержащая N строк и M столбцов (см. 1.1):

$$\begin{pmatrix} a_{11} & \dots & a_{1m} \\ \dots & a_{ij} & \dots \\ a_{n1} & \dots & a_{nm} \end{pmatrix}, \quad (1.1)$$

где позиция каждого элемента матрицы задаётся индексами i и j , отвечающими за положение в строке и столбце соответственно [1].

1.1 Стандартный алгоритм умножения матриц

В стандартном (классическом) алгоритме умножения матриц каждый член результирующей матрицы является скалярным произведением элемента строки первой матрицы и соответствующего элемента столбца второй. Таким образом, умножение двух матриц размера $N \times M$ и $M \times Q$ имеет вид (см. 1.2):

$$\begin{pmatrix} a_{11} & \dots & a_{1m} \\ \dots & \dots & \dots \\ a_{n1} & \dots & a_{nm} \end{pmatrix} \begin{pmatrix} b_{11} & \dots & b_{1q} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mq} \end{pmatrix} = \begin{pmatrix} c_{11} & \dots & c_{1q} \\ \dots & \dots & \dots \\ c_{n1} & \dots & c_{nq} \end{pmatrix}, \quad (1.2)$$

где каждый элемент результирующей матрицы имеет вид (см. 1.3) [2]:

$$c_{ij} = \sum_{k=0}^M a_{ik} * b_{kj}. \quad (1.3)$$

1.2 Алгоритм Винограда

Метод Винограда вносит ряд улучшений в стандартный алгоритм. Так как скалярное произведение двух векторов $u(u1, u2, u3, u4)$ и $v(v1, v2, v3, v4)$, имеющее вид (см. 1.4):

$$u * v = u1 * v1 + u2 * v2 + u3 * v3 + u4 * v4 \quad (1.4)$$

можно переписать как (см. 1.5):

$$u * v = (u1 + v2) * (u2 + v1) + (u3 + v4) * (u4 + v3) - u1 * u2 - u3 * u4 - v1 * v2 - v3 * v4. \quad (1.5)$$

Таким образом, часть значений можно вычислить заранее, что сокращает количество операций умножения [3].

1.3 Алгоритм Винограда с оптимизацией

Исходный алгоритм Винограда можно дополнительно оптимизировать.

Задание по варианту:

- снять индекс в 1-ой части алгоритма;
- заменить $k++$ на $k+=2$.

Первое подзадание предполагает избавление от массива произведений пар элементов первой матрицы. Для этого можно вычитать эти произведения из соответствующих элементов результирующей матрицы в цикле с её заполнением, уменьшая тем количество циклов.

Замена $k++$ на $k+=2$ уменьшает количество умножений, так как теперь обращение к элементам выглядит как $matr[k][i]$, а не $matr[2k][i]$.

Вывод

В данном разделе мной был проведён анализ теоретических материалов, объясняющих принцип работы алгоритмов умножения матриц, а также оптимизирован алгоритм Винограда согласно варианту.

2 Конструкторская часть

2.1 Описание алгоритмов

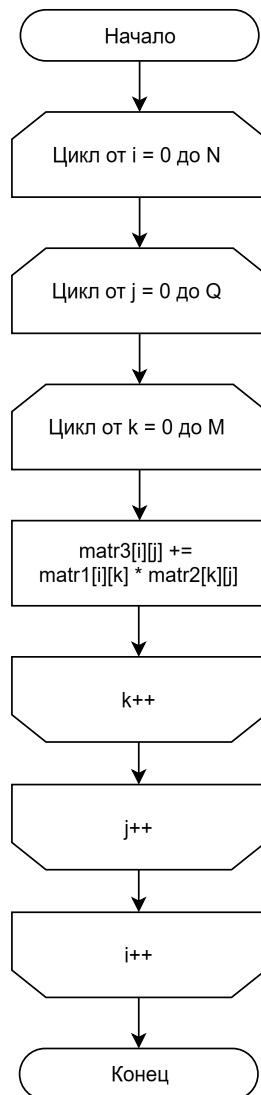


Рисунок 2.1 — Схема стандартного алгоритма

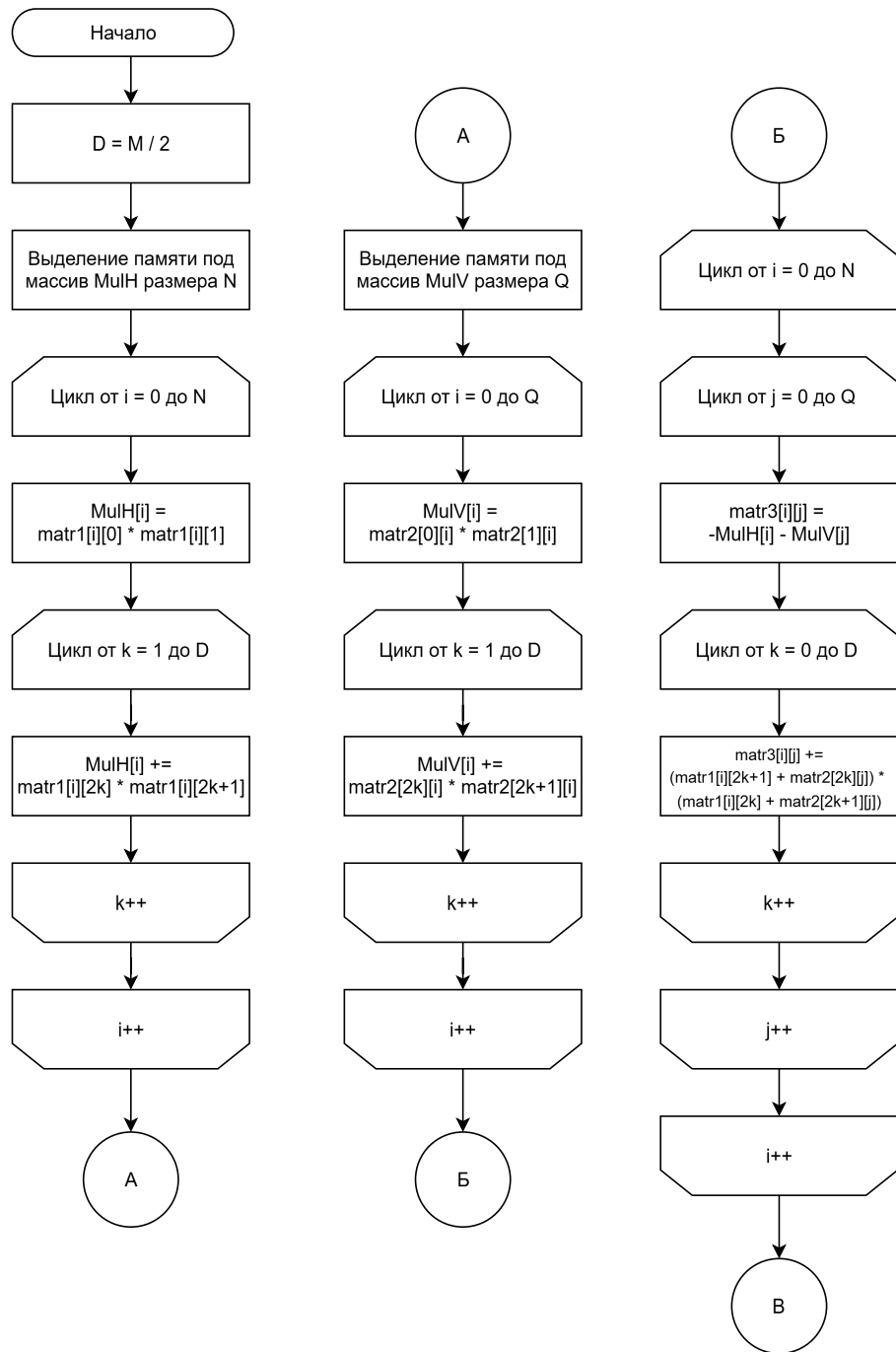


Рисунок 2.2 — Схема алгоритма Винограда

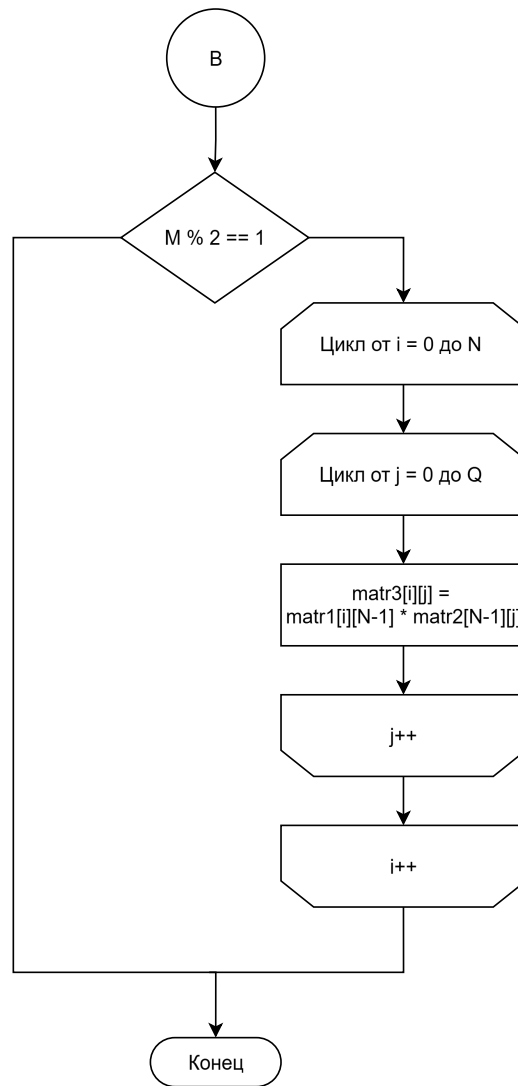


Рисунок 2.3 — Схема алгоритма Винограда

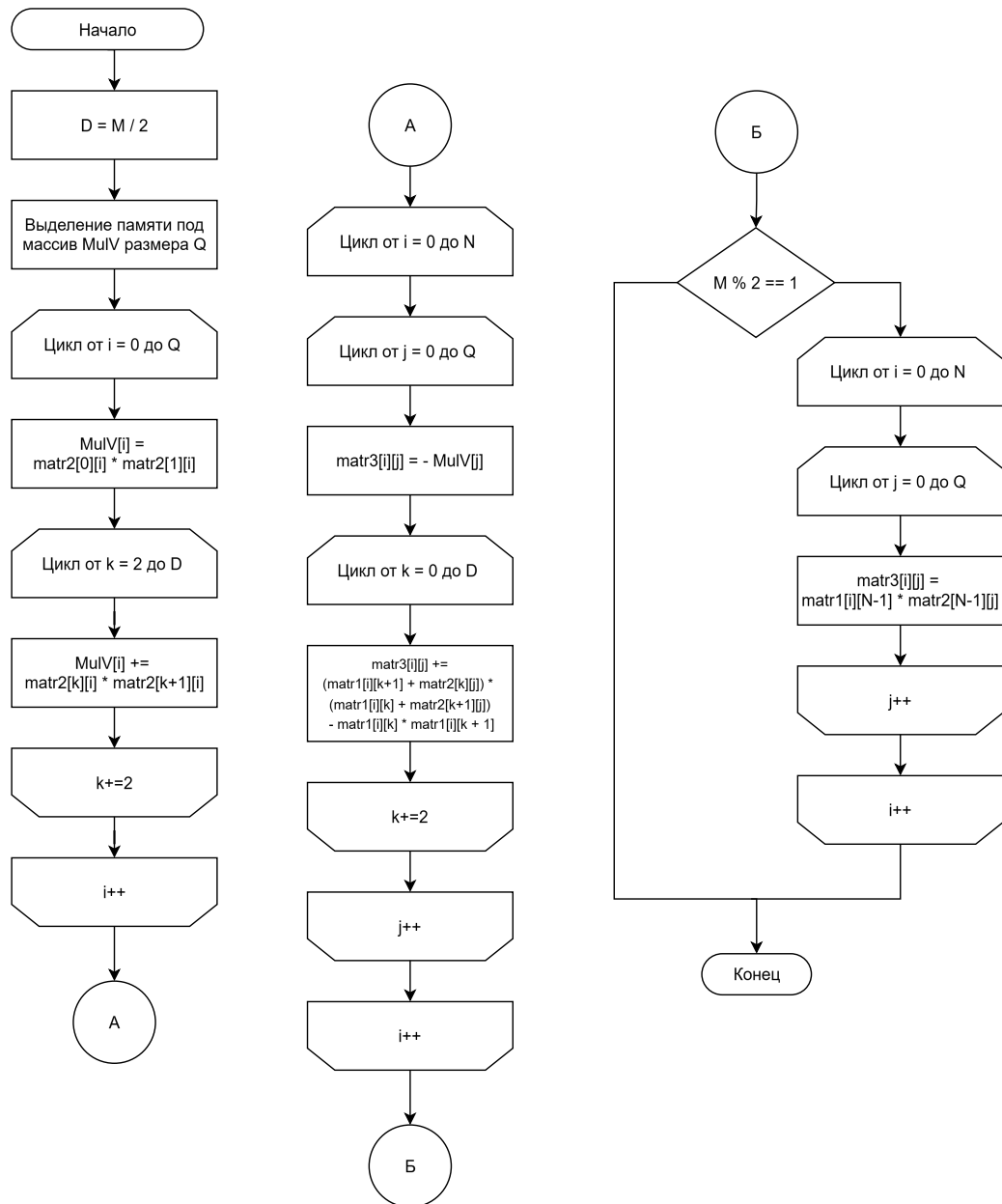


Рисунок 2.4 — Схема алгоритма Винограда с оптимизацией

2.2 Расчёт трудоёмкости алгоритмов

Введём модель вычислений для определения теоретической трудоёмкости описанных ранее алгоритмов.

Таблица 2.1 — Цены базовых операций.

Операции	Цена
$+, -, =, ++, --, \ll, \gg$	1
$*, /, \%, *=, /=$	2
$==, >=, <=, !=, >, <, \&\&, \parallel$	1
$[], \{ \}$	1

Таблица 2.2 — Трудоёмкость алгоритмов.

Алгоритм	Трудоёмкость ЛС	Трудоёмкость ХС
Стандартный	$2 + 2N + 2NQ + 10MNQ$	$2 + 2N + 2NQ + 10MNQ$
Виноград	$12 - 7N - 9Q + 6NM + 9NQ + 13.5NMQ$	$14 - 5N - 9Q + 6NM + 18NQ + 13.5NMQ$
Виноград с оптимизацией	$10 + Q + 2.25MQ + 2N + 7NQ + 13.5NMQ$	$13 + Q + 2.25MQ + 4N + 16NQ + 13.5NMQ$

Вывод

В данном разделе мной были построены схемы алгоритмов и вычислена их трудоёмкость с помощью введённой модели вычислений.

3 Технологическая часть

3.1 Реализация алгоритмов

Для реализации алгоритмов использовался язык программирования C++, так как он поддерживает статическую типизацию и имеет возможность отключения сборщика мусора. Отладка происходила в среде разработки «Visual Studio 2022».

```
1 int std_matrmul(std::vector<std::vector<int>>& matr1, std::vector<std::vector<int>>& matr2,
2   std::vector<std::vector<int>>& matr3,
3   int n, int m, int r, int q){
4   int res = 0;
5   if (m == r)
6     for (int i = 0; i < n; i++){
7       for (int j = 0; j < q; j++){
8         matr3[i][j] = 0;
9         for (int k = 0; k < m; k++)
10            matr3[i][j] += matr1[i][k] * matr2[k][j];
11       }
12     else
13       res = 1;
14   return res;
15 }
```

Листинг 1 – Стандартный алгоритм умножения матриц

```
1 int winograd(std::vector<std::vector<int>>& matr1, std::vector<std::vector<int>>& matr2,
2   std::vector<std::vector<int>>& matr3,
3   int n, int m, int r, int q){
4   int res = 0;
5   if (m == r){
6     int d = m / 2;
7
8     std::vector<int> row_coef(m, 0);
9     for (int i = 0; i < n; i++){
10      row_coef[i] = matr1[i][0] * matr1[i][1];
11      for (int k = 1; k < d; k++)
12        row_coef[i] += matr1[i][2 * k] * matr1[i][2 * k + 1];
13    }
14
15    std::vector<int> col_coef(q, 0);
16    for (int i = 0; i < q; i++){
17      col_coef[i] = matr2[0][i] * matr2[1][i];
18      for (int k = 1; k < d; k++)
19        col_coef[i] += matr2[2 * k][i] * matr2[2 * k + 1][i];
20    }
21
22    for (int i = 0; i < n; i++){
23      for (int j = 0; j < q; j++){
24        matr3[i][j] = -row_coef[i] - col_coef[j];
25        for (int k = 0; k < d; k++)
26          matr3[i][j] += (matr1[i][2 * k + 1] + matr2[2 * k][j]) * (matr1[i][2 * k]
27            + matr2[2 * k + 1][j]);
28      }
29    }
30  }
```

```

27         }
28
29         if (m % 2 == 1){
30             for (int i = 0; i < n; i++)
31                 for (int j = 0; j < q; j++)
32                     matr3[i][j] += matr1[i][m - 1] * matr2[r - 1][j];
33         }
34     }
35     else
36         res = 1;
37
38     return res;
39 }

```

Листинг 2 – Алгоритм Винограда умножения матриц

```

1  int winograd_optimized(std::vector<std::vector<int>>& matr1, std::vector<std::vector<int>>&
   matr2, std::vector<std::vector<int>>& matr3,
2  int n, int m, int r, int q){
3  int res = 0;
4
5  if (m == r){
6      int d = m / 2;
7
8      std::vector<int> col_coef(q, 0);
9      for (int i = 0; i < q; i++){
10         col_coef[i] = matr2[0][i] * matr2[1][i];
11         for (int k = 2; k < d; k += 2)
12             col_coef[i] += matr2[k][i] * matr2[k + 1][i];
13     }
14
15     for (int i = 0; i < n; i++){
16         for (int j = 0; j < q; j++){
17             matr3[i][j] = -col_coef[j];
18             for (int k = 0; k < d; k += 2){
19                 matr3[i][j] += -matr1[i][k] * matr1[i][k + 1] + (matr1[i][k + 1] +
20                     matr2[k][j]) * (matr1[i][k] + matr2[k + 1][j]);
21             }
22         }
23     }
24
25     if (m % 2 == 1){
26         for (int i = 0; i < n; i++)
27             for (int j = 0; j < q; j++)
28                 matr3[i][j] += matr1[i][m - 1] * matr2[r - 1][j];
29     }
30     else
31         res = 1;
32
33     return res;
34 }

```

Листинг 3 – Алгоритм Винограда умножения матриц с оптимизацией

3.2 Тестирование

Таблица 3.1 — Модульные тесты для стандартного алгоритма.

Вход	Выход
$ \begin{array}{c} 4\ 2\ 2\ 4 \\ \begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \begin{pmatrix} 3 & 4 & 2 & 5 \\ 6 & 4 & 3 & 2 \end{pmatrix} \end{array} $	$ \begin{pmatrix} 24 & 20 & 13 & 16 \\ 42 & 36 & 23 & 30 \\ 33 & 28 & 18 & 23 \\ 51 & 44 & 28 & 37 \end{pmatrix} $

Таблица 3.2 — Модульные тесты для алгоритма Винограда.

Вход	Выход
$ \begin{array}{c} 4\ 2\ 2\ 4 \\ \begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \begin{pmatrix} 3 & 4 & 2 & 5 \\ 6 & 4 & 3 & 2 \end{pmatrix} \end{array} $	$ \begin{pmatrix} 24 & 20 & 13 & 16 \\ 42 & 36 & 23 & 30 \\ 33 & 28 & 18 & 23 \\ 51 & 44 & 28 & 37 \end{pmatrix} $
$ \begin{array}{c} 4\ 3\ 3\ 4 \\ \begin{pmatrix} 2 & 3 & 3 \\ 4 & 5 & 5 \\ 3 & 4 & 4 \\ 5 & 6 & 6 \end{pmatrix} \begin{pmatrix} 3 & 4 & 2 & 5 \\ 6 & 4 & 3 & 2 \\ 7 & 3 & 2 & 1 \end{pmatrix} \end{array} $	$ \begin{pmatrix} 45 & 29 & 19 & 19 \\ 77 & 51 & 33 & 35 \\ 61 & 40 & 26 & 27 \\ 93 & 62 & 40 & 43 \end{pmatrix} $

Таблица 3.3 — Модульные тесты для алгоритма Винограда с оптимизацией.

Вход	Выход
$ \begin{array}{c} 4\ 2\ 2\ 4 \\ \begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \begin{pmatrix} 3 & 4 & 2 & 5 \\ 6 & 4 & 3 & 2 \end{pmatrix} \end{array} $	$ \begin{pmatrix} 24 & 20 & 13 & 16 \\ 42 & 36 & 23 & 30 \\ 33 & 28 & 18 & 23 \\ 51 & 44 & 28 & 37 \end{pmatrix} $
$ \begin{array}{c} 4\ 3\ 3\ 4 \\ \begin{pmatrix} 2 & 3 & 3 \\ 4 & 5 & 5 \\ 3 & 4 & 4 \\ 5 & 6 & 6 \end{pmatrix} \begin{pmatrix} 3 & 4 & 2 & 5 \\ 6 & 4 & 3 & 2 \\ 7 & 3 & 2 & 1 \end{pmatrix} \end{array} $	$ \begin{pmatrix} 45 & 29 & 19 & 19 \\ 77 & 51 & 33 & 35 \\ 61 & 40 & 26 & 27 \\ 93 & 62 & 40 & 43 \end{pmatrix} $

Вывод

В данном разделе мной были реализованы алгоритмы умножения матриц и проведено их успешное тестирование.

4 Исследовательская часть

4.1 Замеры процессорного времени

Замеры проводились на ноутбуке Redmibook Pro 14 2022:

- процессор Intel i7-12650H (архитектура x86-64, 10 ядер по 2.3 ГГц);
- оперативная память 16 Гб (DDR4, 5200 МГц);
- операционная система Windows 11 Home 24H2.

Для замеров использовалась функция `clock()` из библиотеки `ctime` [4, с. 288].

Таблица 4.1 — Некоторые замеры процессорного времени для квадратных матриц.

Размер	Стандартный	Виноград	Виноград с оптимизацией
2	0.0005	0.0005	0.0005
3	0.0005	0.0005	0.0010
9	0.0120	0.0085	0.0055
10	0.0105	0.0105	0.0090
49	1.0710	1.0265	0.6920
50	1.0885	1.1060	0.7635

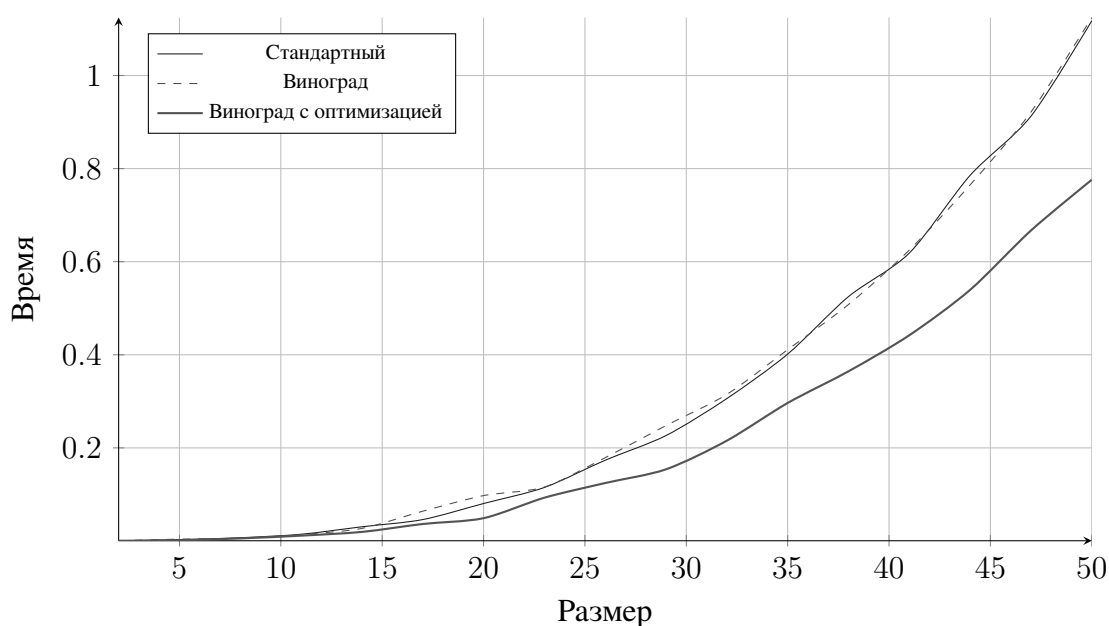


Рисунок 4.1 — График зависимости процессорного времени от размера матрицы

Вывод

В данном разделе были проведены замеры процессорного времени для каждой реализации из предыдущего раздела. Полученные на основе этих замеров графики иллюстрируют рассчитанные ранее трудоёмкости рассматриваемых в работе алгоритмов.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были описаны алгоритмы умножения матриц, введена модель вычислений и с её помощью была вычислена трудоёмкость каждого из алгоритмов. Также были написаны реализации алгоритмов и выполнены замеры процессорного времени для лучшего и худшего случаев для каждой из реализаций.

Рассчитанная ранее теоретическая оценка трудоёмкости алгоритмов соответствует полученным на практике значениям. Реализация с наименьшим значением трудоёмкости – реализация алгоритма Винограда с оптимизацией.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ильин, В.А. Линейная алгебра: Учеб.Для вузов – 4-е изд. / В.А. Ильин, Э.Г. Позняк – М.Наука. Физматлит, 1999 – 256с. – Текст: непосредственный.
2. Стандартный метод умножения матриц. /sciencedirect.com [Электронный ресурс] Режим доступа: <https://www.sciencedirect.com/topics/computer-science/matrix-multiplication> (дата обращения: 22.09.25).
3. Karakadov B. Ultra-Fast Matrix Multiplication: An Empirical Analysis of Highly Optimized Vector Algorithms /cs.stanford.edu [Электронный ресурс] Режим доступа: https://cs.stanford.edu/people/boyko/pubs/MatrixMult_SURJ_2004.pdf (дата обращения: 01.11.2025).
4. C17 standard (ISO/IEC 9899:2018) [стр. 288]. [Электронный ресурс] Режим доступа: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (дата обращения: 29.09.2025).